

ECO-LLM: LLM-based Edge Cloud Optimization

Kunal Rao
NEC Laboratories America, Inc.
New Jersey, USA

Giuseppe Coviello
NEC Laboratories America, Inc.
New Jersey, USA

Priscilla Benedetti*
NEC Laboratories America, Inc.
New Jersey, USA

Ciro Giuseppe De Vita†
NEC Laboratories America, Inc.
New Jersey, USA

Gennaro Mellone‡
NEC Laboratories America, Inc.
New Jersey, USA

Srimat Chakradhar
NEC Laboratories America, Inc.
New Jersey, USA

ABSTRACT

AI/ML techniques have been used to solve systems problems, but their applicability to customize solutions on-the-fly has been limited. Traditionally, any customization required manually changing the AI/ML model or modifying the code, configuration parameters, application settings, etc. This incurs too much time and effort, and is very painful. In this paper, we propose a novel technique using Generative Artificial Intelligence (GenAI) technology, wherein instructions can be provided in natural language and actual code to handle any customization is automatically generated, integrated and applied on-the-fly. Such capability is extremely powerful since it makes customization of application settings or solution techniques super easy. Specifically, we propose *ECO-LLM* (LLM-based Edge Cloud Optimization), which leverages Large Language Models (LLM) to dynamically adjust placement of application tasks across edge and cloud computing tiers, in response to changes in application workload, such that insights are delivered quickly with low cost of operation (systems problem). Our experiments with real-world video analytics applications i.e. face recognition, human attributes detection and license plate recognition show that *ECO-LLM* is able to automatically generate code on-the-fly and adapt placement of application tasks across edge and cloud computing tiers. We note that the trigger workload (to switch between edge and cloud) for *ECO-LLM* is exactly the same as the baseline (manual) and actual placement performed by *ECO-LLM* is only slightly different i.e. on average (across 2 days) only 1.45% difference in human attributes detection and face recognition, and 1.11% difference in license plate recognition. Although we tackle this specific systems problem in this paper, our proposed GenAI-based technique is applicable to solve other systems problems too.

*Work done as an intern at NEC Laboratories America, Inc. while being affiliated with Vrije Universiteit Brussel - Brussels, Belgium and University of Perugia - Perugia, Italy

†Work done as an intern at NEC Laboratories America, Inc. while being affiliated with University of Napoli Parthenope - Napoli, Italy

‡Work done as an intern at NEC Laboratories America, Inc. while being affiliated with University of Napoli Parthenope - Napoli, Italy

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

AI4Sys '24, June 3–7, 2024, Pisa, Italy

© 2024 Copyright held by the owner/author(s). Publication rights licensed to Association for Computing Machinery.

ACM ISBN 979-8-4007-0652-3/24/06.

<https://doi.org/10.1145/3660605.3660941>

CCS CONCEPTS

• **Computer systems organization** → *n-tier architectures; Real-time system architecture*; • **Computing methodologies** → *Online learning settings; Information extraction; Natural language generation*.

KEYWORDS

Large Language Models (LLM), Generative Artificial Intelligence (GenAI), Machine Learning (ML), Customization, Optimization, Edge computing, Cloud computing, Video Analytics

ACM Reference Format:

Kunal Rao, Giuseppe Coviello, Priscilla Benedetti, Ciro Giuseppe De Vita, Gennaro Mellone, and Srimat Chakradhar. 2024. *ECO-LLM: LLM-based Edge Cloud Optimization* In *Workshop on AI For Systems (AI4Sys '24)*, June 3–7, 2024, Pisa, Italy. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3660605.3660941>

1 INTRODUCTION

AI/ML techniques have proven to be very useful in handling systems problems [4][13][22][9][13]. In all these, the required models, sophisticated algorithms, etc. need to be built apriori and incorporated into the system software at the time of deployment. If some model or heuristic needs to change during operation, then one has to rebuild the model or modify the code or application settings and re-deploy. Such customization takes a lot of time and effort. Especially, when these customizations are not straight forward and the system problem spans across multiple tiers of computing.

Modern infrastructure consists of multiple layers of computing infrastructure, including computing capability on devices, computing capability at the edge and on-premises, computing capability in MEC (Multi-access edge computing) [8] and computing in the cloud. There are pros and cons of computing at each tier e.g. computing at the edge delivers quick response but there is limited compute power, whereas computing in the cloud has lot of compute power, but it comes at a cost, due to the pay-per-use cost model of cloud computing. Applications which require real-time performance tend to use edge computing for faster response times, but when compute power at the edge falls short (due to increase in application workload), applications can leverage the high performance compute available in the cloud. For microservices-based real-time stream processing applications, dynamically adapting the placement of microservices across edge and cloud computing tiers, in response to changes in the application workload, enables quick response with low cost of operation. Logic for such dynamic placement decisions can be fed into the system by manually profiling the application and adding the code to adapt placement settings. However, if the application

or the underlying infrastructure or the placement logic/heuristics change, then one has to manually make the necessary changes and re-deploy. [18] proposes a graph partitioning based solution which takes application-specific information to dynamically determine appropriate placement of microservices across multiple tiers of computing. But this is still very inflexible, time consuming, requires too much effort and is quite painful. Products like AnB [19] enable simplified deployment and operation, but making custom changes to the applications and underlying mechanisms is still quite difficult.

In recent times, Generative Artificial Intelligence (GenAI) technology has become quite popular and content generated from large language models (LLMs) [5, 7, 12, 17] is quite appealing. OpenAI released ChatGPT to the public in November, 2022 [15] and since then, apart from asking simple and basic questions, people have started using it for a variety of tasks including writing stories, poems and songs [20], answering complex standardized test questions [11], generating ideas [24], translating between several languages, summarizing text [21], writing and debugging computer programs [27], etc. Given the right prompts, it is possible to extract very high quality output from LLMs and guide it into accurately performing the task at hand. Such prompts can also lead the LLM to generate actual working code. In this paper, we leverage this capability of LLM to empower users to easily customize their solutions. Specifically, we propose *ECO-LLM*, which is a LLM-based middleware for Edge Cloud Optimization. Users can just provide customization instructions in natural language and under-the-hood, working code is automatically generated, integrated and applied on-the-fly by *ECO-LLM*. Such method of delivering customizability to users is quite different than traditional approaches where configuration parameters are exposed to users and they can set appropriate values. In our proposed method, instead of providing user-settable configuration parameters, we allow users to just write in natural language and behind the scene, the necessary code is automatically generated and “retrofitted” into the application. Although, we choose the systems problem of placement of application tasks across edge and cloud computing tiers, our GenAI-based technique can be applied to solve other systems problems as well.

In summary, our key contributions are:

- We present Generative Artificial Intelligence (GenAI) based architecture to solve systems problems, wherein users are empowered to easily customize their application settings or solutions by giving instructions in natural language and actual code is automatically generated, integrated and applied on-the-fly under-the-hood, without any manual effort.
- We propose a novel middleware, *ECO-LLM*, which leverages Large Language Models (LLMs) to handle dynamic placement of microservices on a multi-tiered computing infrastructure, in response to changes in the application workload, such that insights are delivered quickly in a cost-efficient manner (systems problem).
- With three real-world video analytics applications i.e. face recognition, human attributes detection and license plate recognition, we show that the trigger workloads (to switch between edge and cloud) identified by *ECO-LLM* exactly match the *baseline* (manual). The actual placement performed

by *ECO-LLM* on average (across 2 days) shows only 1.45% difference for human attributes and face recognition, and 1.11% difference in license plate recognition.

The rest of the paper is organized as follows. We discuss related work in Section 2. In Section 3, we show the need for a system to automatically adjust placement of microservices on multiple tiers of computing. Then, in Section 4, we discuss how we leverage LLM to provide custom instructions in natural language and generate code on-the-fly. Further in Section 5, we present a practical LLM-based system (*ECO-LLM*) to dynamically adjust placement of microservices on a multi-tiered computing infrastructure. Finally, we report our results in Section 6 and conclude in Section 7.

2 RELATED WORK

ML techniques have been applied for resource allocation [4][13] and job scheduling [3][28] in various use cases. For example, in [22], RL-based Markov Decision Processes are leveraged to optimize resource allocation for VMs in a vehicular cloud. [9] on the other hand, exploits RL to manage resource allocation in the Internet of Things (IoT) using Quality of Experience (QoE) as reward for updating states. A similar use case is considered in [13], where task offloading policies for IoT users is modeled by a Markov decision process and each user is considered as an agent making a series of task offloading decisions with the goal of minimizing the system cost. All the considered works rely on RL techniques to handle job scheduling, resource allocation and task offloading problems. These solutions require a certain level of operational effort to carefully define the model and train it for the specific environment. In our work, we leverage Generative Artificial Intelligence (GenAI) technology, specifically Large Language Models (LLM), where we can express the problem in natural language and LLM comes up with a solution. Once the LLM receives the right instructions, it does not require a lot of operational effort and is portable across different deployment scenarios.

Natural Language Processing (NLP) has been used to solve various systems problems as well. For example, [25] [26] use NLP to mine database tuning hints from web documents and apply it to database configuration. Such configuration tuning problem in the context of microservices-based applications is studied by Gagan et al. in [23]. They propose to use LLM as part of the configuration tuning pipeline for large scale cloud services. Their scope, however, is quite large, which includes configuration of individual microservices, and they haven’t actually evaluated in a real-world scenario. Similar to these works, *ECO-LLM* also tunes configuration (in this case, placement configuration), but the scope for *ECO-LLM* is very specific and *ECO-LLM* has actually been evaluated in a real-world scenario. To the best of our knowledge, *ECO-LLM* is the first to leverage LLM for dynamic adjustment of placement of microservices across multiple tiers of computing, in response to changes in application workload, such that application insights are delivered quickly and with low cost of operation.

3 MOTIVATION

Fig. 1 shows three microservices-based, representative video analytics application pipelines i.e. human attributes detection, face recognition and license plate recognition that we consider. We use 2 tiers of computing i.e. *edge* and *cloud*. For *edge*, we use two different

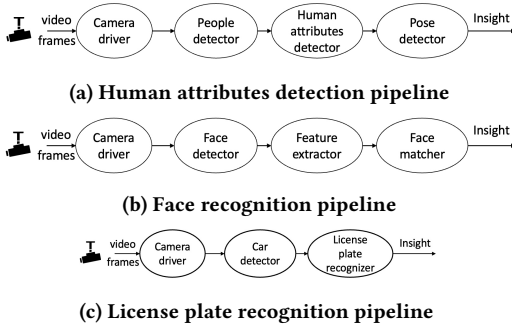


Figure 1: Representative video analytics application pipelines

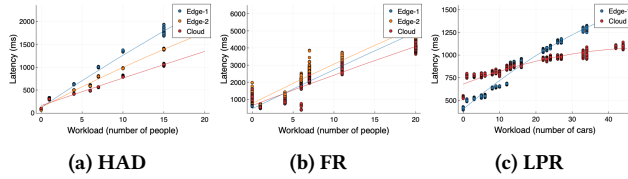


Figure 2: Characterization of application latency (HAD: Human Attributes Detection, FR: Face Recognition, LPR: License Plate Recognition)

edge machines denoted as *Edge-1* (equipped with AMD Ryzen 9 5900HX 8-core processor) and *Edge-2* (equipped with AMD Ryzen 9 5900 12-Core processor), while for *cloud* we use c4.8xlarge VM instance in AWS cloud. Regarding placement of various microservices, “camera-driver”, “people detector”, “face detector” and “car detector” microservices always run on the edge, while the other downstream microservices (“human attributes detector”, “pose detector”, “feature extractor”, “face matcher” and “license plate recognizer”) can run either on the *edge* or *cloud*.

Fig. 2 shows the characterization of latency for each of these application pipelines. Here, “Edge-1” and “Edge-2” refers to the placement where all microservices of the application pipeline are executed on “Edge-1” and “Edge-2”, respectively, and “Cloud”, refers to the placement where, “human attributes detector” and “pose detector” (for human attributes detection), “feature extractor” and “face matcher” (for face recognition), and “license plate recognizer” (for license plate recognition) are executed on the cloud VM instance. Note that only one application pipeline with a specific placement runs at any given time i.e. we do not run multiple application pipelines concurrently. We note that for these application pipelines, the latency characterization for the two *edge* machines is different (shown in Fig. 2a and Fig. 2b), and for lower workloads, *edge* delivers faster response (low latency), while for higher workloads, *cloud* delivers faster response.

We note that the *trigger* point (intersection point for the workload where *cloud* starts performing better than the *edge*) for same application on different “edge” infrastructure (“Edge-1” and “Edge-2”) varies, and also, on the same infrastructure, the trigger point for different applications is different. Thus, to know when to use *edge* or *cloud* i.e. at which workload, limited compute on *edge* is sufficient and when high compute performance in the cloud is needed, one has to profile and characterize the application on the specific infrastructure and generate such graphs. Then, based on this data, *trigger* point can be identified. Doing this each time for every new

You
You are an optimizer who decides placement on multi-tiered computing infrastructure consisting of edge computing and cloud computing. Cost for using edge computing is 0 while cost for using cloud computing is 1. When real-time video analytics application runs on this computing infrastructure, there is latency to receive insights. As the workload increases, which is indicated by the number of people in the video, using only edge computing increases latency. At this point, using cloud computing resources will reduce the latency to receive insights. Your goal is to suggest placement such that latency as well as cost of running the video analytics application is low.

For a workload, your job is to understand application latency when run on edge only and when run on edge + cloud and build a model, which will recommend placement, which will be given as either “edge” or “cloud” in JSON format for the specific workload seen in front of the video camera.

Do not assume any model for latency. You have to build it based on the data you see. Model for cost is as follows: when only “edge” is used, cost is 0 and when “cloud” is used cost is 1. Once you see data for a specific workload for “edge” and “cloud”, you should remember it and next time for that workload you should suggest the appropriate placement. Usage of “cloud” should be triggered only if the improvement in latency is over 20 percent, otherwise continue using “edge”. Remember, your job is to suggest placement for a given workload, such that both latency and cost is low.

Now, when prompt is “Recommend placement for workload: <workload>” you should return the JSON format with suggested placement. The key in JSON should be “placement” and value can only be “edge” or “cloud”. Always give output in this JSON format. Use the data you have already seen and if you do not have data for the requested workload, then give placement for which you do NOT have data and once you have data for “edge” and “cloud”, use the latency and cost model to suggest the right placement. Remember that for each new workload you see, you should try both, edge and cloud placement.

When the prompt is “Latency on <infrastructure> for workload: <workload> is <latency>”, record the latency number for the latency model you are building. Here <infrastructure> is either “edge” or “cloud”.

ChatGPT
I’ve set up a system to record and analyze the latency for different workloads on both edge and cloud computing infrastructures. You can now provide me with latency data or ask for placement recommendations for specific workloads. [2-]

Figure 3: Placement guidance

application on every new infrastructure is practically infeasible. Therefore, there is a need for automating this and figuring out the *trigger* point for the specific application on a given *edge* + *cloud* multi-tiered computing infrastructure.

4 CUSTOM INSTRUCTIONS AND ON-THE-FLY CODE GENERATION

In order to automate the placement of microservices in response to changes in the application workload, we leverage LLM to guide the placement. Specifically, through custom instructions, we show how LLM is able to generate code on-the-fly and apply it to perform dynamic placement as the application workload changes. This technique paves way for others to incorporate such LLM-based code generation to customize and solve other systems problems as well.

The first instruction to ChatGPT v4, shown in Fig. 3, is a very detailed prompt, which gives all the necessary information to the LLM in order to guide it into making appropriate placement decisions. We use the basic principles of *prompt engineering* discussed in [16], [10], [14], etc. and craft the “prompt” such that the LLM is able to perform data-driven placement recommendation, considering latency and cost of operation. In this prompt, we provide guidance such as - when to use *cloud* and when to stay on the *edge*. Note that this guidance is generic and not specific to a particular application or infrastructure. The guidance given works for any application and on any *edge* + *cloud* infrastructure. As part of this prompt, we also tell the LLM how the cost model is and how it should build latency model based on the provided latency data and then using these 2 models, how it should respond back (along with format specification) with the placement recommendation. The response from ChatGPT v4, shown in Fig. 3 indicates that the instructions were understood and that it is ready to process the data as desired and respond back with placement recommendation. Fig. 4 shows

```

# Initialize a dictionary to store the latency data for both edge and cloud computing based
# on workload.
# The structure will be: {workload: {"edge": latency_edge, "cloud": latency_cloud}}
latency_data = {}

def record_latency(infrastructure, workload, latency):
    """
    Record the latency for a specific infrastructure (edge/cloud) and workload.
    """
    if workload not in latency_data:
        latency_data[workload] = {}
    latency_data[workload][infrastructure] = latency

def recommend_placement(workload):
    """
    Recommend the placement for a given workload based on the recorded latency and cost
    """
    # If data for both edge and cloud is available, use the model to decide; otherwise, return
    # the infrastructure where data is missing.
    if workload in latency_data and "edge" in latency_data[workload] and "cloud" in latency_data[workload]:
        latency_edge = latency_data[workload]["edge"]
        latency_cloud = latency_data[workload]["cloud"]
        # If the improvement in latency by using cloud is over 20%, recommend cloud, else
        # recommend edge.
        if (latency_edge - latency_cloud) / latency_edge > 0.20:
            return {"placement": "cloud"}
        else:
            return {"placement": "edge"}
    else:
        # If data for both infrastructures is not available, default to recommending edge
        # until cloud data is obtained.
        return {"placement": "edge" if "edge" not in latency_data.get(workload, {}) else "cloud"}

# The function 'record_latency' and 'recommend_placement' are now defined and ready to be
# used based on the prompt.

```

Figure 4: LLM generated code



Figure 5: Placement recommendation

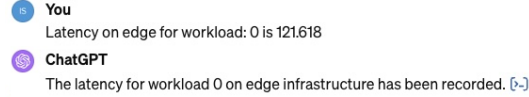


Figure 6: Impact of placement recommendation

the python code automatically generated and integrated by LLM to handle the placement according to the provided instructions. Clear, unambiguous and detailed instructions in the prompt lead the LLM into generating bug-free code. Note that currently the language used for code generation is Python, but it could very well be in any other language as well. Also, although the code looks simple enough, the fact that it was automatically generated based on instructions written in natural language is quite appealing. This technique is quite encouraging and promising, and can be applied to solve even more complex and challenging systems problems.

The second instruction, shown in Fig. 5, is a single line prompt, which gives the workload to the LLM and asks for placement recommendation. We note that the response given back is in the JSON format with the right key ("placement") and value ("edge" or "cloud") as instructed in the first prompt.

The third instruction, shown in Fig. 6, reports the latency observed for the recommended placement (given in second instruction response) back to the LLM. This prompt essentially gives feedback to the LLM so that it can know the impact of the given recommendation. This helps LLM in refining future recommendations.

Only the first instruction is given *one-time* at the beginning, while the other two are given *repeatedly* (at specified intervals) to

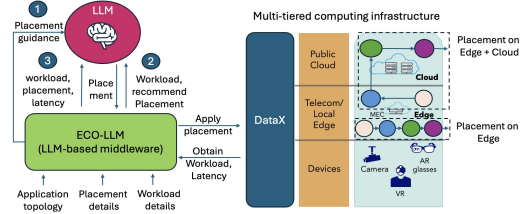


Figure 7: System design

the LLM during operation, so that it can make appropriate data-driven placement decisions. These instructions can be tailored by users and customized as per the needs and characteristics of their applications. Since users understand their applications and corresponding workload and processing needs better, they can apply different optimization and scheduling techniques. For example, for applications with people-focus, the workload depends on the number of people in the video scene, whereas for applications with car-focus, the workload depends on the number of cars in the video scene. This way of exposing customizability does not restrict users to specific configuration parameters (as exposed by traditional methods). Rather, they can create and define new parameters on their own and that too by just writing in plain natural language. Through such capability, entirely new settings or optimization algorithms can be fed by users into *ECO-LLM*, which makes this GenAI-based system design very powerful.

5 SYSTEM DESIGN AND IMPLEMENTATION

Using the custom instructions discussed in Section 4, we design a practical system called *ECO-LLM*, which leverages DataX [6] to deploy and run microservices-based applications on a multi-tiered computing infrastructure. Fig. 7 shows the system design for *ECO-LLM*. In order to dynamically place application microservices, *ECO-LLM* needs to know certain application related details. These include (a) Application topology (b) Placement details and (c) Workload details. All these are specified in a single YAML file and provided to *ECO-LLM*. Application topology is the list of various interconnected microservices that comprise the application pipeline. Placement details gives the "modes" of placement i.e. "edge" or "cloud" mode, indicating the placement for various microservices on *edge* or *edge + cloud* computing tiers. This also includes a default placement mode. Workload details specifies the manner in which application workload should be fetched. This is typically part of the output of an intermediate microservice in the pipeline e.g. number of people (workload) for human attributes detection application is in the output of "people detector" microservice. With these application related details, *ECO-LLM* is equipped with all necessary information to dynamically place microservices with the help of LLM and DataX on the multi-tiered computing infrastructure.

Fig. 8 shows the flowchart with the various steps taken by *ECO-LLM* in order to dynamically place microservices. At the beginning, *ECO-LLM* uses the *first* instruction to give guidance to the LLM regarding the manner in which LLM should perform placement and what considerations it should take into account¹. Then, *ECO-LLM* deploys the application on DataX with the default placement. After the application starts running, *ECO-LLM* fetches the application

¹*ECO-LLM* uses OpenAI's APIs to send prompts and receive responses

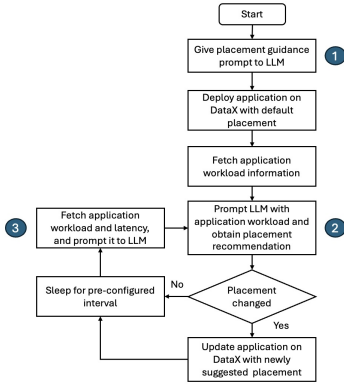


Figure 8: ECO-LLM flowchart

You
What is the workload when cloud is preferred over the edge? Give response with just a single number for the workload.

ChatGPT
2

Figure 9: ECO-LLM Trigger workload

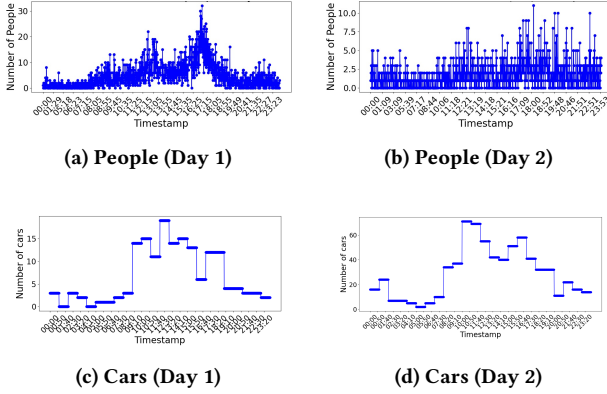


Figure 10: Variation in workload

workload using *dataxctl* command line tool provided by DataX and uses the *second* instruction to obtain placement recommendation from LLM. *ECO-LLM* then checks if the placement recommended by the LLM is the same or different than the current placement. If it has changed, then *ECO-LLM* updates the application pipeline using the newly suggested placement. If not, *ECO-LLM* lets the application run as is. Then, *ECO-LLM* sleeps for a pre-configured interval (typically 30 seconds or 1 minute). After that, *ECO-LLM* fetches the workload and the application latency (again, using *dataxctl* command line tool) and uses the *third* instruction to report the latency to LLM. After reporting latency, *ECO-LLM* again provides the application workload and requests for placement recommendation. The *second* and *third* instructions are repeated throughout the lifetime of the application. This way, *ECO-LLM* periodically checks the status of the application workload, the latency corresponding to the placement, and leverages LLM to dynamically make placement decisions, which are applied via DataX to run the application microservices on the multi-tiered computing infrastructure.

6 EXPERIMENTAL RESULTS

We present our experimental results in this section. We initially verify that the responses provided by ChatGPT v4 are correct and

Placement method	HAD	FR	LPR
Baseline	2	36	34
ECO-LLM	2	36	34

Table 1: Trigger Workloads

	HAD		FR		LPR	
TPD	1440		1440		1440	
	Day1	Day2	Day1	Day2	Day1	Day2
TIP Edge (E)	28	10	0	0	10	0
TIP Cloud (C)	2	2	30	12	10	12
TIP (E+C)	30	12	30	12	20	12
Percentage (%)	2.08	0.83	2.08	0.83	1.39	0.83

Table 2: Results obtained using ECO-LLM (TPD: Total Placement Decisions, TIP: Total Incorrect Placements)

then apply it for automatic placement. We then compare it against a baseline placement, which is the one that a human operator would have set after manually profiling the application. We use the same applications and infrastructure discussed in Section 3.

6.1 Manual

To manually verify whether ChatGPT v4 is responding with correct answers, we initially gave the placement guidance (first instruction) and then requested placement when workload was 0. The response was “edge”. We then reported the latency for “edge” obtained from the characterization of human attributes application (shown in Section 3). We again asked the placement for workload 0, and this time it responded with “cloud” because it doesn’t have data for cloud. We then provided the latency for cloud which is higher than edge. After this, we again ask placement for workload 0 and this time ChatGPT correctly recommended “edge” placement as it is faster and cheaper. We repeated this for workload 1, where edge latency was higher than cloud latency, but the difference was less than 20%. ChatGPT correctly understood the instructions and recommended “edge” placement. Then we repeated this with workload 2, where the difference between edge and cloud latency is greater than 20%, and we noted that this time ChatGPT correctly recommended “cloud” placement. Thus, we manually verified that ChatGPT v4 produces correct output. Later, we asked the trigger workload (to switch between edge and cloud), as shown in Fig. 9, and ChatGPT correctly responded as 2.

6.2 Automatic

In order to apply *ECO-LLM* on real-world data, we used two different public datasets recorded at the city of Melbourne. The first one shows the count of people [1] and the second shows the traffic count [2] at different locations across multiple days. We choose a specific location and use the data for 2 days (weekday and weekend). Fig. 10 shows the profile for people and cars, which represents the workload for the three applications under consideration.

Table 1 shows the trigger workloads (to switch between edge and cloud) used for both, *baseline* placement and *ECO-LLM*. We observe that *ECO-LLM* automatically identified the exact same trigger workloads. This indicates that LLM was able to correctly analyze the data and figure out the condition for change in placement. Table 2 shows the placement decisions made by *ECO-LLM* in comparison

to *baseline* placement. For each day, there are total 1440 placement decisions, denoted by Total Placement Decisions (TPD)². We count the total number of times *ECO-LLM* incorrectly recommends “edge” placement, when *baseline* placement chooses cloud (denoted by Total Incorrect Placements (TIP) Edge) and the total number of times *ECO-LLM* incorrectly recommends “cloud” placement when *baseline* placement chooses edge (denoted as TIP Cloud). Then, we sum this and show the Total Incorrect Placements (TIP) on edge and cloud together, and calculate the percentage when placement recommended by *ECO-LLM* is different from the placement chosen by *baseline* placement. Such differences will happen with *ECO-LLM* by design, because *ECO-LLM* tries all placement options each time a new workload is seen, so that it can make proper data-drive placement decisions. Our results show that on average (across 2 days) the difference is only 1.45% for human attributes detection and face recognition, and 1.11% for license plate recognition, which is quite low. The few times when the recommended placements are incorrect, the latency takes a hit. In our experiments, the maximum hit in the latency observed for any workload is the difference between the edge and cloud characterization curves shown in Fig. 2.

7 CONCLUSION

In this paper, we propose Generative Artificial Intelligence (GenAI) based technology to solve systems problems, wherein natural language is used to customize the application settings or solution methodology, and the necessary code generation and integration automatically takes place under-the-hood. We target the systems problem of efficient placement of tasks across multi-tiered computing infrastructure and propose *ECO-LLM*, which leverages LLM to generate and apply code in order to adapt placement of microservices, in response to change in application workload. Our experimental results with three real-world video analytics applications i.e. human attributes detection, face recognition and license plate recognition, show that the trigger workloads (to switch between edge and cloud) identified by *ECO-LLM* accurately matches the *baseline* (manual) placement and the actual placement performed by *ECO-LLM* is only slightly different. The difference in placement is on average (across 2 days) only 1.45% for human attributes detection and face recognition, and 1.11% for license plate recognition. Our proposed technique is general and can be applied to solve other systems problems as well.

REFERENCES

- [1] 2022. Pedestrian Counting System - Past Hour (counts per minute). https://melbournetestbed.opendatasoft.com/explore/dataset/pedestrian-counting-system-past-hour-counts-per-minute/information/?sort=-location_id
- [2] 2022. Traffic Count Vehicle Classification 2014-2017. <https://melbournetestbed.opendatasoft.com/explore/dataset/traffic-count-vehicle-classification-2014-2017/information/>
- [3] Mehmet Emin Aydin and Ercan Öztemel. 2000. Dynamic job-shop scheduling using reinforcement learning agents. *Robotics Auton. Syst.* 33 (2000), 169–178.
- [4] Ali Belgacem, Saïd Mahmoudi, and Maria Kihl. 2022. Intelligent multi-agent reinforcement learning model for resources allocation in cloud computing. *Journal of King Saud University - Computer and Information Sciences* 34, 6, Part A (2022), 2391–2404. <https://doi.org/10.1016/j.jksuci.2022.03.016>
- [5] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. [arXiv:cs.CL/2005.14165](https://arxiv.org/abs/2005.14165)
- [6] Giuseppe Coviello, Kunal Rao, Murugan Sankaradas, and Srimat Chakradhar. 2022. DataX: A System for Data eXchange and Transformation of Streams. In *Intelligent Distributed Computing XIV*. Springer International Publishing, Cham, 319–329.
- [7] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. [arXiv:cs.CL/1810.04805](https://arxiv.org/abs/1810.04805)
- [8] Abderrahime Filali, Amine Abouamar, Soumaya Cherkaoui, Abdellatif Kobbane, and Mohsen Guizani. 2020. Multi-Access Edge Computing: A Survey. *IEEE Access* 8 (2020), 197017–197046. <https://doi.org/10.1109/ACCESS.2020.3034136>
- [9] Keke Gai and Meikang Qiu. 2018. Optimal resource allocation using reinforcement learning for IoT content-centric services. *Applied Soft Computing* 70 (2018), 12–21. <https://doi.org/10.1016/j.asoc.2018.03.056>
- [10] Google. August 8, 2023. Prompt Engineering for Generative AI. <https://developers.google.com/machine-learning/resources/prompt-eng>
- [11] Rebecca Heilweil. Dec 7, 2022. AI is finally good at stuff, and that’s a problem. <https://www.vox.com/recode/2022/12/7/23498694/ai-artificial-intelligence-chat-gpt-openai>
- [12] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Ves Stoyanov, and Luke Zettlemoyer. 2019. BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension. [arXiv:cs.CL/1910.13461](https://arxiv.org/abs/1910.13461)
- [13] Xiaolan Liu, Jiadong Yu, Jian Wang, and Yue Gao. 2020. Resource Allocation With Edge Computing in IoT Networks via Machine Learning. *IEEE Internet of Things Journal* 7, 4 (2020), 3415–3426. <https://doi.org/10.1109/JIOT.2020.2970110>
- [14] OpenAI. [n.d.]. Prompt Engineering. <https://platform.openai.com/docs/guides/prompt-engineering>
- [15] OpenAI. Nov 30, 2022. Introducing ChatGPT. <https://openai.com/blog/chatgpt>
- [16] James Phoenix and Mike Taylor. July 2024. Prompt Engineering for Generative AI. O’Reilly Media, Inc., ISBN: 9781098153434. <https://www.oreilly.com/library/view/prompt-engineering-for/9781098153427/>
- [17] Alec Radford and Karthik Narasimhan. 2018. Improving Language Understanding by Generative Pre-Training. <https://api.semanticscholar.org/CorpusID:49313245>
- [18] Kunal Rao, Giuseppe Coviello, Wang-Pin Hsiung, and Srimat Chakradhar. 2021. ECO: Edge-Cloud Optimization of 5G applications. In *2021 IEEE/ACM 21st International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*. 649–659. <https://doi.org/10.1109/CCGrid51090.2021.00078>
- [19] Kunal Rao, Murugan Sankaradas, Giuseppe Coviello, Ciro Giuseppe De Vita, Gennaro Mellone, Wang-Pin Hsiung, and Srimat Chakradhar. 2023. ANB: Application-in-a-Box to Rapidly Deploy and Self-optimize 5G Apps. In *2023 IEEE International Conference on Smart Computing (SMARTCOMP)*. 82–89. <https://doi.org/10.1109/SMARTCOMP58114.2023.00028>
- [20] Aaron Reich. Dec 27, 2022. ChatGPT: What is the new free AI chatbot? - explainer. <https://www.jpost.com/business-and-innovation/tech-and-start-ups/article-725910>
- [21] Elizabeth Rider. April 6, 2023. How ChatGPT Will Dramatically Change the Influencer Space. <https://www.entrepreneur.com/science-technology/how-chatgpt-will-dramatically-change-the-influencer-space/448386>
- [22] Mohammad A. Salahuddin, Ala Al-Fuqaha, and Mohsen Guizani. 2016. Reinforcement learning for resource provisioning in the vehicular cloud. *IEEE Wireless Communications* 23, 4 (2016), 128–135. <https://doi.org/10.1109/MWC.2016.7553036>
- [23] Gagan Somashekar and Rajat Kumar. 2023. Enhancing the Configuration Tuning Pipeline of Large-Scale Distributed Applications Using Large Language Models (Idea Paper). In *Companion of the 2023 ACM/SPEC International Conference on Performance Engineering (ICPE ’23 Companion)*. Association for Computing Machinery, New York, NY, USA, 39–44. <https://doi.org/10.1145/3578245.3585032>
- [24] Josh Folk Tojin T. Eapen, Daniel J. Finkenshtadt and Lokesh Venkataswamy. July 2023. How Generative AI Can Augment Human Creativity. <https://hbr.org/2023/07/how-generative-ai-can-augment-human-creativity>
- [25] Immanuel Trummer. 2021. The case for NLP-enhanced database tuning: towards tuning tools that “read the manual”. *Proc. VLDB Endow.* 14, 7 (mar 2021), 1159–1165. <https://doi.org/10.14778/3450980.3450984>
- [26] Immanuel Trummer. 2022. DB-BERT: A Database Tuning Tool that “Reads the Manual”. In *Proceedings of the 2022 International Conference on Management of Data (SIGMOD ’22)*. Association for Computing Machinery, New York, NY, USA, 190–203. <https://doi.org/10.1145/3514221.3517843>
- [27] Liam Tung. Jan 26, 2023. ChatGPT can write code. Now researchers say it’s good at fixing bugs, too. <https://www.zdnet.com/article/chatgpt-can-write-code-now-researchers-say-its-good-at-fixing-bugs-too/>
- [28] Wei Zhang and Thomas G. Dietterich. 1995. A Reinforcement Learning Approach to job-shop Scheduling. In *IJCAI*.

²Placement decision is taken every minute. So, there are 60 x 24 (1440) placement decisions in a 24 hour period. For missing entries in dataset, we repeat the previous value until next entry is observed. We conduct experiments separately for each day.